ErthaLoka Planetary Performance System (PPS)

Part 1: Formulas, Algorithms & Calculations

Technical Deep-Dive Documentation v2.0

Classification: Technical Implementation Reference **Version:** 2.0 **Date:** December 2025

TABLE OF CONTENTS

1. Overview
2. Normalization Functions
3. PPS Scoring Algorithm
4. ESV Calculation
5. Carbon Credit Calculation
6. Data Quality Score (DQS)
7. Grading Scale (AAA-CCC)
8. ESV Projections
9. Complete Formula Reference

---

1. OVERVIEW

This document provides the complete mathematical and algorithmic specifications for the ErthaLoka Planetary Performance System (PPS). All formulas are extracted directly from the production codebase.

1.1 Core Calculation Flow

```
Raw Satellite Data
       │
       ▼
┌──────────────────┐
│  Normalization   │  ← 6 different functions based on metric type
│   (0-100 scale)  │
└──────────┬───────┘
         │
         ▼
┌──────────────────┐
│ Category Score   │  ← Weighted average within each pillar (A,B,C,D,E)
│  Calculation     │
└──────────┬───────┘
         │
         ▼
┌──────────────────┐
│ Overall PPS      │  ← Ecosystem-adaptive weighted aggregation
│     Score        │
└──────────┬───────┘
         │
         ▼
┌──────────────────┐       ┌──────────────────┐
│  ESV Multiplier  │       │     Grading      │
│   Calculation    │       │   (AAA-CCC)      │
└──────────┬───────┘       └──────────────────┘
         │
```

```
        ▼
┌─────────────────┐
│   Final ESV     │
│   (USD/ha/yr)   │
└─────────────────┘
```

---

2. NORMALIZATION FUNCTIONS

2.1 Overview

The PPS uses **6 distinct normalization functions** to convert raw metric values to a standardized 0-100 score. The choice of function depends on the ecological relationship between raw values and ecosystem health.

Source File: `/Hello-/planetary_health_query/utils/normalization.py`

2.2 Linear Normalization

When to use: Higher raw values indicate better ecosystem health (monotonically positive relationship).

Formula:

```
S = [(V - V_min) / (V_max - V_min)] × 100
```

Python Implementation:

```python
def linear_normalize(value: float, v_min: float, v_max: float) -> float:
    """
    Linear normalization: S = [(V - V_min) / (V_max - V_min)] * 100

    Higher raw values produce higher scores.
    Used for: NDVI, EVI, Tree Cover, FPAR, Visibility
    """
    if v_max == v_min:
        return 50.0  # Avoid division by zero

    # Clamp value to range
    clamped = max(v_min, min(v_max, value))
    score = ((clamped - v_min) / (v_max - v_min)) * 100
    return max(0.0, min(100.0, score))
```

Applied to:

Metric	V_min	V_max	Description
NDVI	-0.1	0.9	Vegetation health
EVI	0.0	1.0	Enhanced vegetation
Tree Cover	0	100	Forest coverage %
FPAR	0.0	0.95	Photosynthetically active radiation
Visibility	1	50	Atmospheric clarity (km)

Example Calculation:

```
Input: NDVI = 0.65
V_min = -0.1, V_max = 0.9
```

```
Score = [(0.65 - (-0.1)) / (0.9 - (-0.1))] × 100
      = [0.75 / 1.0] × 100
      = 75.0
```

2.3 Inverse Linear Normalization

When to use: Lower raw values indicate better ecosystem health (monotonically negative relationship).

Formula:

```
S = [(V_max - V) / (V_max - V_min)] × 100
```

Python Implementation:

```python
def inverse_linear_normalize(value: float, v_min: float, v_max: float) -> float:
    """
    Inverse Linear normalization: S = [(V_max - V) / (V_max - V_min)] * 100

    Lower raw values produce higher scores.
    Used for: AOD, AQI, Forest Loss, Human Modification, Nightlights
    """
    if v_max == v_min:
        return 50.0

    clamped = max(v_min, min(v_max, value))
    score = ((v_max - clamped) / (v_max - v_min)) * 100
    return max(0.0, min(100.0, score))
```

Applied to:

Metric	V_min	V_max	Description
AOD	0.0	1.5	Aerosol Optical Depth
AQI	0	300	Air Quality Index
Forest Loss	0	10	% forest lost
Human Modification	0.0	1.0	Human impact index
Distance to Water	0	50	km to nearest water

Example Calculation:

```
Input: AQI = 75
V_min = 0, V_max = 300

Score = [(300 - 75) / (300 - 0)] × 100
      = [225 / 300] × 100
      = 75.0
```

2.4 Sigmoid Normalization

When to use: Indicator exhibits threshold effects with diminishing returns at high values (S-shaped response curve).

Formula:

```
S = 100 / [1 + exp(-k × (V - V_mid))]
```

Where:

- `k` = steepness parameter (scaled by range)
- `V_mid` = inflection point (midpoint of transition)

Python Implementation:

```python
def sigmoid_normalize(
    value: float,
    v_min: float,
    v_max: float,
    k: float = 0.5,
    v_mid: Optional[float] = None
) -> float:
    """
    Sigmoid normalization: S = 100 / [1 + exp(-k * (V - V_mid))]

    Produces S-shaped curve centered at V_mid.
    Used for: LAI, Biomass, Canopy Height, UV Index
    """
    if v_mid is None:
        v_mid = (v_min + v_max) / 2

    # Scale k based on the range to maintain consistent steepness
    range_val = v_max - v_min
    if range_val > 0:
        k_scaled = k * (10 / range_val)
    else:
        k_scaled = k

    try:
        exponent = -k_scaled * (value - v_mid)
        # Prevent overflow
        if exponent > 700:
            return 0.0
        elif exponent < -700:
            return 100.0
        score = 100 / (1 + math.exp(exponent))
        return max(0.0, min(100.0, score))
    except (OverflowError, ValueError):
        return 50.0
```

Applied to:

Metric	V_min	V_max	V_mid	k	Description
LAI	0.0	7.0	3.5	0.5	Leaf Area Index
Biomass	0	400	200	0.5	Above-ground biomass (Mg/ha)
Canopy Height	0	40	20	0.5	Forest height (m)
UV Index	0	11	5.5	0.5	UV radiation intensity

Example Calculation:

```
Input: Biomass = 250 Mg/ha
V_min = 0, V_max = 400, V_mid = 200, k = 0.5

k_scaled = 0.5 × (10 / 400) = 0.0125
exponent = -0.0125 × (250 - 200) = -0.625
Score = 100 / [1 + exp(-0.625)]
      = 100 / [1 + 0.535]
      = 100 / 1.535
      = 65.1
```

2.5 Inverse Sigmoid Normalization

When to use: Lower values are better but with diminishing returns at extremes.

Formula:

```
S = 100 - sigmoid(V)
```

Python Implementation:

```python
def inverse_sigmoid_normalize(
    value: float,
    v_min: float,
    v_max: float,
    k: float = 0.5,
    v_mid: Optional[float] = None
) -> float:
    """
    Inverse Sigmoid: S = 100 - sigmoid(V)

    Used for: Population Density, Night Lights
    """
    return 100.0 - sigmoid_normalize(value, v_min, v_max, k, v_mid)
```

Applied to:

Metric	V_min	V_max	V_mid	k	Description
Population	0	5000	500	0.3	People per km^2
Nightlights	0	63	20	0.3	Radiance ($nW/cm^2/sr$)

2.6 Gaussian (Optimal Range) Normalization

When to use: Intermediate values are optimal; both extremes indicate poor ecosystem health.

Formula:

```
S = 100 × exp[-(V - V_opt)² / (2 × σ²)]
```

Where:

- `V_opt` = optimal value (peak of Gaussian)
- `σ` = standard deviation (width of optimal range)

Python Implementation:

```python
def gaussian_normalize(
    value: float,
    v_opt: float,
    sigma: float,
    v_min: Optional[float] = None,
    v_max: Optional[float] = None
) -> float:
    """
    Gaussian/Optimal normalization: S = 100 * exp[-(V - V_opt)^2 / (2 * sigma^2)]

    Maximum score at optimal value, decreasing symmetrically.
    Used for: LST (25°C optimal), Soil Moisture (30% optimal)
    """
    if v_min is not None and v_max is not None:
        value = max(v_min, min(v_max, value))

    if sigma == 0:
        return 100.0 if value == v_opt else 0.0

    try:
        exponent = -((value - v_opt) ** 2) / (2 * sigma ** 2)
        score = 100 * math.exp(exponent)
        return max(0.0, min(100.0, score))
    except (OverflowError, ValueError):
        return 0.0
```

Applied to:

Metric	V_min	V_max	V_opt	σ	Description
LST	15	45	25	8	Land Surface Temp (°C)
Soil Moisture	0.05	0.45	0.30	0.12	Volumetric water content

Example Calculation:

```
Input: LST = 30°C
V_opt = 25, σ = 8

exponent = -[(30 - 25)² / (2 × 8²)]
        = -[25 / 128]
        = -0.195

Score = 100 × exp(-0.195)
      = 100 × 0.823
      = 82.3
```

2.7 Centered Normalization

When to use: Zero or near-zero values are optimal (e.g., drought indices where 0 indicates normal conditions).

Formula:

```
S = 100 × [1 - (|V| / V_max)]
```

Python Implementation:

```python
def centered_normalize(value: float, v_max: float) -> float:
    """
    Centered normalization: S = 100 * [1 - (|V| / V_max)]

    Maximum score at V=0, decreasing with distance from zero.
    Used for: Drought Index, Evaporative Stress
    """
    if v_max == 0:
        return 100.0 if value == 0 else 0.0

    abs_ratio = abs(value) / abs(v_max)
    score = 100 * (1 - abs_ratio)
    return max(0.0, min(100.0, score))
```

Applied to:

Metric	V_max	Description
Drought Index	2.0	PDSI scale (-2 to +2)
Evaporative Stress	1.0	ESI scale (-1 to +1)

Example Calculation:

```
Input: Drought Index = -0.8 (mild drought)
V_max = 2.0

Score = 100 × [1 - (|-0.8| / 2.0)]
      = 100 × [1 - 0.4]
      = 100 × 0.6
      = 60.0
```

---

3. PPS SCORING ALGORITHM

3.1 Category/Pillar Score Calculation

Source File: `/Hello-/planetary_health_query/utils/scoring.py`

Formula:

```
C_j = Σ(w_i × S_i) / Σw_i
```

Where:

- `C_j` = Category score for pillar j
- `w_i` = Weight of indicator i within the category
- `S_i` = Normalized score of indicator i (0-100)

Python Implementation:

```python
def calculate_category_score(
    category_id: str,
    metrics: Dict[str, Any]
) -> Tuple[Optional[float], Dict[str, float]]:
    """
    Calculate weighted score for a single category/pillar.
```

```python
    Formula: C_j = Sum(w_i * S_i) / Sum(w_i)
    """
    if not metrics:
        return None, {}

    weighted_sum = 0.0
    total_weight = 0.0
    metric_scores = {}

    for metric_name, metric_data in metrics.items():
        # Get raw value
        if isinstance(metric_data, dict):
            value = metric_data.get("value")
        else:
            value = metric_data

        if value is None:
            continue

        # Check if metric belongs to this category
        params = PHI_METRIC_PARAMS.get(metric_name)
        if not params:
            continue

        metric_category = params.get("category")
        if metric_category and metric_category != category_id:
            continue

        # Normalize the value
        score = normalize_metric(metric_name, value)
        if score is None:
            continue

        metric_scores[metric_name] = round(score, 2)

        # Apply weight (skip zero-weight informational metrics)
        weight = params.get("weight", 0.25)
        if weight > 0:
            weighted_sum += score * weight
            total_weight += weight

    if total_weight == 0:
        return None, metric_scores

    category_score = weighted_sum / total_weight
    return round(category_score, 2), metric_scores
```

3.2 Overall PPS Score Calculation

Formula:

```
PPS = W_A × A + W_B × B + W_C × C + W_D × D + W_E × E
```

Where `W_x` are ecosystem-specific weights that sum to 1.0

Python Implementation:

```python
def calculate_overall_score(
    pillar_scores: Dict[str, float],
    ecosystem_type: str = "default"
) -> Optional[float]:
    """
    Calculate overall PPS score using ecosystem-adaptive weights.

    Formula: PPS = W_A*A + W_B*B + W_C*C + W_D*D + W_E*E
    """
    if not pillar_scores:
        return None

    # Get ecosystem-specific weights
    ecosystem_config = ECOSYSTEM_CATEGORY_WEIGHTS.get(
        ecosystem_type,
        ECOSYSTEM_CATEGORY_WEIGHTS["default"]
    )

    weights = {k: v for k, v in ecosystem_config.items() if k in ["A", "B", "C", "D", "E"]}

    weighted_sum = 0.0
    total_weight = 0.0

    for pillar_id, score in pillar_scores.items():
        if score is None:
            continue

        weight = weights.get(pillar_id, 0.20)
        weighted_sum += score * weight
        total_weight += weight

    if total_weight == 0:
        return None

    # Normalize by total weight used (handles missing pillars)
    return round(weighted_sum / total_weight, 2)
```

3.3 Ecosystem-Adaptive Weights

Source: `/Hello-/planetary_health_query/core/config.py`

```python
ECOSYSTEM_CATEGORY_WEIGHTS = {
    "tropical_forest": {
        "A": 0.10, "B": 0.25, "C": 0.35, "D": 0.15, "E": 0.15,
        "description": "Emphasizes carbon storage and biodiversity"
    },
    "mangrove": {
        "A": 0.10, "B": 0.20, "C": 0.30, "D": 0.25, "E": 0.15,
        "description": "Emphasizes carbon and degradation (water stress)"
    },
    "grassland_savanna": {
        "A": 0.15, "B": 0.30, "C": 0.15, "D": 0.25, "E": 0.15,
        "description": "Emphasizes biodiversity and degradation"
    },
    "wetland": {
```

```
        "A": 0.10, "B": 0.25, "C": 0.20, "D": 0.30, "E": 0.15,
        "description": "Emphasizes degradation (water/soil conditions)"
    },
    "agricultural": {
        "A": 0.20, "B": 0.25, "C": 0.10, "D": 0.25, "E": 0.20,
        "description": "Balanced with emphasis on air quality and services"
    },
    "urban_green": {
        "A": 0.30, "B": 0.25, "C": 0.10, "D": 0.15, "E": 0.20,
        "description": "Strong emphasis on atmospheric quality"
    },
    "default": {
        "A": 0.20, "B": 0.20, "C": 0.20, "D": 0.20, "E": 0.20,
        "description": "Equal weighting for mixed/unknown ecosystems"
    }
}
```

Weight Distribution Visualization:

```
Tropical Forest:     A[▓▓    ] B[▓▓▓▓▓ ] C[▓▓▓▓▓▓▓] D[▓▓▓  ] E[▓▓▓  ]
Mangrove:            A[▓▓    ] B[▓▓▓▓  ] C[▓▓▓▓▓▓ ] D[▓▓▓▓▓] E[▓▓▓  ]
Grassland:           A[▓▓▓   ] B[▓▓▓▓▓▓] C[▓▓▓    ] D[▓▓▓▓▓] E[▓▓▓  ]
Wetland:             A[▓▓    ] B[▓▓▓▓▓ ] C[▓▓▓▓   ] D[▓▓▓▓▓▓] E[▓▓▓  ]
Agricultural:        A[▓▓▓▓  ] B[▓▓▓▓▓ ] C[▓▓     ] D[▓▓▓▓▓] E[▓▓▓▓ ]
Urban Green:         A[▓▓▓▓▓▓] B[▓▓▓▓▓ ] C[▓▓     ] D[▓▓▓  ] E[▓▓▓▓ ]
Default:             A[▓▓▓▓  ] B[▓▓▓▓  ] C[▓▓▓▓   ] D[▓▓▓▓ ] E[▓▓▓▓ ]
                     0%   10%   20%   30%
```

---

4. ESV CALCULATION

4.1 PHI-to-ESV Multiplier

Source File: `/src/utils/esvCalculation.ts` and `/Hello-/planetary_health_query/utils/scoring.py`

Formula:

```
M_PHI = [(PHI - 50) / 100] × k × [1 + α × ln(PHI / 50)]
```

Where:

- `k = 0.6` (base sensitivity factor)
- `α = 0.15` (logarithmic acceleration factor)
- `PHI` = PPS score (0-100)

TypeScript Implementation:

```typescript
/**
 * Calculate PHI-to-ESV multiplier
 * Formula: M_PHI = [(PHI − 50) / 100] × k × [1 + α × ln(PHI / 50)]
 */
export function calculatePHIMultiplier(phiScore: number): number {
  if (phiScore <= 0) return -0.30; // Cap at -30% for zero/negative scores

  const k = 0.6;
  const alpha = 0.15;
```

```
  const baseTerm = (phiScore - 50) / 100;
  const logTerm = 1 + alpha * Math.log(phiScore / 50);

  const multiplier = baseTerm * k * logTerm;

  // Clamp between -0.30 and +0.50
  return Math.max(-0.30, Math.min(0.50, multiplier));
}
```

Python Implementation:

```
def calculate_phi_esv_multiplier(phi_score: float) -> Optional[float]:
    """
    Calculate PHI-to-ESV multiplier.

    Formula: M_PHI = [(PHI - 50) / 100] * k * [1 + alpha * ln(PHI / 50)]
    """
    if phi_score is None or phi_score <= 0:
        return None

    k = PHI_ESV_CONSTANTS.get("k", 0.6)
    alpha = PHI_ESV_CONSTANTS.get("alpha", 0.15)

    phi_clamped = max(phi_score, 1.0)

    try:
        base = (phi_clamped - 50) / 100
        ratio = phi_clamped / 50
        if ratio <= 0:
            return None

        log_factor = 1 + alpha * math.log(ratio)
        multiplier = base * k * log_factor
        return round(multiplier, 4)
    except (ValueError, ZeroDivisionError, OverflowError):
        return None
```

Multiplier Effect Table:

PPS Score	Multiplier	ESV Adjustment	Interpretation
20	-0.22	-22%	Severely degraded
30	-0.15	-15%	Highly degraded
40	-0.08	-8%	Moderately degraded
50	0.00	0%	Baseline reference
60	+0.08	+8%	Healthy
70	+0.16	+16%	Very healthy
80	+0.25	+25%	Excellent
90	+0.35	+35%	Pristine
95	+0.41	+41%	Reference condition

4.2 Baseline ESV Values

Source: `/src/utils/esvCalculation.ts`

```
const ECOSYSTEM_BASELINE_ESV: Record<string, number> = {
  tropical_forest: 5382,
  temperate_forest: 3137,
  mangrove: 12696,
  coral_reef: 352249,
  wetland: 25682,
  grassland_savanna: 2871,
  agricultural: 5567,
  urban_green: 6661,
  desert: 591,
  default: 3500,
};
```

4.3 Service-Specific ESV Breakdown

```
const SERVICE_VALUES: Record<string, Record<string, number>> = {
  tropical_forest: {
    carbon_sequestration: 800,
    water_regulation: 600,
    air_quality: 300,
    biodiversity_habitat: 1500,
    recreation_cultural: 400,
    pollination: 200,
    erosion_control: 400,
  },
  wetland: {
    carbon_sequestration: 600,
    water_regulation: 8000,
    air_quality: 200,
    biodiversity_habitat: 3000,
    recreation_cultural: 1000,
    pollination: 500,
    erosion_control: 2000,
  },
  // ... other ecosystems
};
```

4.4 Regional Adjustment Factors (RAF)

```
const REGIONAL_FACTORS: Record<string, number> = {
  india_tier1: 0.45,
  india_tier2_3: 0.35,
  india_rural: 0.25,
  southeast_asia_urban: 0.50,
  africa_subsaharan: 0.20,
  europe_north_america: 1.00,
  default: 0.35,
};
```

4.5 Complete ESV Calculation

Formula:

```
ESV_adjusted = ESV_baseline × (1 + M_PHI) × Regional_Factor
```

TypeScript Implementation:

```typescript
export function calculateESVBreakdown(
  phiScore: number,
  ecosystemType: EcosystemType | string = 'default',
  region: string = 'default'
): ESVBreakdownResult {
  const baselineValue = getBaselineESV(ecosystemType);
  const regionalFactor = getRegionalFactor(region);
  const phiMultiplier = calculatePHIMultiplier(phiScore);

  // Get service values for ecosystem type
  const serviceValues = SERVICE_VALUES[ecosystemType] || SERVICE_VALUES.default;

  // Calculate total from services
  const totalServiceValue = Object.values(serviceValues).reduce((sum, val) => sum + val, 0);

  // Apply PHI multiplier and regional factor
  const adjustedTotal = totalServiceValue * (1 + phiMultiplier) * regionalFactor;

  // Build service breakdown
  const services: ServiceValue[] = Object.entries(serviceValues).map(([key, value]) => {
    const adjustedValue = value * (1 + phiMultiplier) * regionalFactor;
    return {
      name: formatServiceName(key),
      value: Math.round(adjustedValue),
      percentage: Math.round((value / totalServiceValue) * 100),
      // ...
    };
  });

  return {
    totalValue: totalServiceValue,
    adjustedValue: Math.round(adjustedTotal),
    phiMultiplier,
    regionalFactor,
    ecosystemType: formatEcosystemTypeName(ecosystemType),
    services,
    baselineValue,
  };
}
```

Example Calculation:

```
Input:
  - PPS Score: 72
  - Ecosystem: Tropical Forest
  - Region: India Tier 2/3

Step 1: Calculate PHI Multiplier
```

```
  M_PHI = [(72 - 50) / 100] × 0.6 × [1 + 0.15 × ln(72/50)]
        = 0.22 × 0.6 × [1 + 0.15 × 0.364]
        = 0.132 × 1.0546
        = 0.139 ≈ +14%

Step 2: Get Baseline ESV
  Baseline = $5,382/ha/year (tropical forest)

Step 3: Apply Adjustments
  ESV_adjusted = 5382 × (1 + 0.139) × 0.35
               = 5382 × 1.139 × 0.35
               = $2,145/ha/year
```

---

5. CARBON CREDIT CALCULATION

5.1 IPCC Tier 1 Methodology

Source: `/Hello-/planetary_health_query/core/engine.py`

Conversion Chain:

```
Biomass (Mg/ha)
    │
    │ × 0.47 (carbon fraction)
    ▼
Carbon Stock (tC/ha)
    │
    │ × 3.67 (CO2/C molecular ratio = 44/12)
    ▼
CO2 Equivalent (tCO2e/ha)
    │
    │ × (DQS/100) (confidence adjustment)
    ▼
Verified CO2 (tCO2e/ha)
    │
    │ × Price ($/tonne)
    ▼
Credit Value (USD/ha)
```

5.2 Carbon Stock Calculation

Formulas:

```
Carbon_Stock = Biomass × 0.47

CO2_Equivalent = Carbon_Stock × 3.67

Total_CO2 = CO2_Equivalent × Area_ha

Verified_CO2 = Total_CO2 × (DQS / 100)

Credit_Value = Verified_CO2 × Price_per_tonne
```

Constants:

- `0.47` = IPCC default carbon fraction of dry biomass
- `3.67` = Molecular weight ratio of CO2 to C (44/12)

- `DQS` = Data Quality Score (0-100)
- Price range: $15-50 per tonne CO2

5.3 Biomass Estimation (Chave et al., 2014)

For field-verified assessments:

Pantropical Allometric Equation:

```
AGB = 0.0673 × (ρ × DBH² × H)^0.976
```

Where:

- `ρ` = Wood density (g/cm³)
- `DBH` = Diameter at breast height (cm)
- `H` = Total tree height (m)
- `AGB` = Above-ground biomass (Mg/ha)

5.4 Below-Ground Biomass

Root-to-Shoot Ratios:

```
BGB = AGB × R

Where R (root-to-shoot ratio):
- Tropical forests: 0.20-0.30
- Temperate forests: 0.30-0.40
- Grasslands: 0.40-0.60
```

Total Biomass:

```
Total_Biomass = AGB + BGB
```

---

6. DATA QUALITY SCORE (DQS)

6.1 DQS Formula

Source File: `/Hello-/planetary_health_query/utils/quality.py`

Formula:

```
DQS = [Σ(w_i × a_i) / Σw_i] × 100
```

Where:

- `w_i` = Criticality weight for metric i
- `a_i` = Availability score:
  - 1.0 if available with good quality
  - 0.5 if available with moderate quality
  - 0.25 if available with poor quality
  - 0.0 if unavailable

6.2 Criticality Weights

Source: `/Hello-/planetary_health_query/core/config.py`

```
CRITICALITY_WEIGHTS = {
    "critical": 1.0,      # NDVI, Tree Cover, Soil Moisture, Human Modification
```

```
    "important": 0.7,     # EVI, AOD, Drought Index, Biomass, Canopy Height
    "supporting": 0.4,    # LAI, FPAR, LST, Population, Night Lights
    "auxiliary": 0.2      # UV Index, Visibility, Distance to Water
}
```

Metric Criticality Assignment:

Tier	Weight	Metrics
Critical	1.0	NDVI, Tree Cover, Soil Moisture, Human Modification
Important	0.7	EVI, AOD, AQI, Drought Index, Biomass, Canopy Height
Supporting	0.4	LAI, FPAR, LST, Population, Nightlights, Forest Loss
Auxiliary	0.2	UV Index, Visibility, Distance to Water, Elevation

6.3 Python Implementation

```python
def calculate_dqs(
    metrics_availability: Dict[str, bool],
    data_quality_flags: Dict[str, str]
) -> float:
    """
    Calculate Data Quality Score using criticality-weighted formula.

    Formula: DQS = [Sum(w_i * a_i) / Sum(w_i)] * 100
    """
    weighted_sum = 0.0
    total_weight = 0.0

    for metric_name, params in PHI_METRIC_PARAMS.items():
        # Get criticality weight
        criticality = params.get("criticality", "supporting")
        weight = CRITICALITY_WEIGHTS.get(criticality, 0.4)

        # Skip zero-weight informational metrics
        if params.get("weight", 0) == 0:
            continue

        # Determine availability score
        is_available = metrics_availability.get(metric_name, False)
        quality = data_quality_flags.get(metric_name, "unavailable")

        if not is_available or quality == "unavailable":
            availability_score = 0.0
        elif quality == "poor":
            availability_score = 0.25
        elif quality == "moderate":
            availability_score = 0.5
        else:  # "good" or "supplemented"
            availability_score = 1.0

        weighted_sum += weight * availability_score
        total_weight += weight
```

```
    if total_weight == 0:
        return 0.0

    return round((weighted_sum / total_weight) * 100, 2)
```

6.4 DQS Thresholds

```
DQS_THRESHOLDS = {
    "category_minimum": 40,      # Minimum DQS for valid category score
    "overall_minimum": 50,       # Minimum DQS for valid overall PHI
    "investment_grade": 70,      # DQS threshold for investment-grade data
    "high_confidence": 85        # DQS threshold for high confidence results
}
```

Confidence Levels:

DQS Range	Level	Use Case
85-100	High Confidence	Carbon credits, regulatory compliance
70-84	Investment Grade	Financial decisions, TNFD disclosure
50-69	Acceptable	Planning, monitoring
40-49	Marginal	Initial screening only
0-39	Low Confidence	Not recommended for decisions

---

7. GRADING SCALE (AAA-CCC)

7.1 Score-to-Grade Mapping

Source File: `/src/types/phi.ts`

```
// Score interpretation (AAA to CCC grading scale)
export type ScoreInterpretation =
  | 'AAA'    // Excellent (86-100)
  | 'AA'     // Very Good (72-85)
  | 'A'      // Good (58-71)
  | 'BBB'    // Above Average (44-57)
  | 'BB'     // Average (30-43)
  | 'B'      // Below Average (16-29)
  | 'CCC'    // Poor (0-15)
  | 'Unavailable';

// Helper function to get grade from score
export const getGradeFromScore = (score: number | null | undefined): ScoreInterpretation => {
  if (score === null || score === undefined) return 'Unavailable';
  if (score >= 86) return 'AAA';
  if (score >= 72) return 'AA';
  if (score >= 58) return 'A';
  if (score >= 44) return 'BBB';
  if (score >= 30) return 'BB';
  if (score >= 16) return 'B';
```

```
  return 'CCC';
};
```

7.2 Grade Definitions

Grade	Score Range	Label	Description	Risk Level
AAA	86-100	Excellent	Pristine/reference ecosystem	Minimal
AA	72-85	Very Good	High-performing, minor limitations	Very Low
A	58-71	Good	Healthy with some stress indicators	Low
BBB	44-57	Above Average	Functional but with degradation	Moderate
BB	30-43	Average	Under stress, recovery uncertain	Elevated
B	16-29	Below Average	Significant degradation	High
CCC	0-15	Poor	Severely degraded, collapse risk	Very High

7.3 Grade Distribution Visualization

```
Score  0    20    40    60    80    100
       |     |     |     |     |     |
  CCC  ├─────┤
    B        ├─────┤
   BB              ├─────┤
  BBB                    ├─────┤
    A                          ├─────┤
   AA                                ├─────┤
  AAA                                      ├─────┤
```

7.4 Institutional Use Cases by Grade

Grade	Bank Lending	Insurance	Investment	Government
AAA-AA	Standard terms	Standard premium	Green bond eligible	Conservation priority
A	Standard terms	Standard premium	Sustainability-linked	Sustainable use
BBB	Enhanced monitoring	Moderate loading	ESG funds possible	Development permitted
BB	Higher collateral	Risk loading	Specialist only	Conditional permits
B	Restricted lending	High loading	Impact investors	Remediation required
CCC	No new exposure	Excluded	Avoid	Emergency intervention

---

8. ESV PROJECTIONS

8.1 Projection Scenarios

Source File: `/src/utils/esvProjection.ts`

```
export const PROJECTION_SCENARIOS: Record<ScenarioName, ProjectionScenario> = {
  conservative: {
    name: 'conservative',
```

```
    label: 'Conservative',
    growthRate: 0.02,     // 2% annual
    carbonRate: 1.5,      // tCO2/ha/year
    description: 'Minimal intervention, natural recovery only',
  },
  moderate: {
    name: 'moderate',
    label: 'Moderate',
    growthRate: 0.05,     // 5% annual
    carbonRate: 3.5,      // tCO2/ha/year
    description: 'Active management, sustainable practices',
  },
  optimistic: {
    name: 'optimistic',
    label: 'Optimistic',
    growthRate: 0.10,     // 10% annual
    carbonRate: 6.0,      // tCO2/ha/year
    description: 'Restoration projects, reforestation',
  },
};
```

8.2 ESV Projection Formula

Formula:

```
ESV(year) = ESV_current × (1 + growth_rate)^year
```

TypeScript Implementation:

```
function calculateYearESV(currentESV: number, growthRate: number, year: number): number {
  return currentESV * Math.pow(1 + growthRate, year);
}
```

8.3 PHI Projection Formula

PHI improves slower than ESV:

```
function calculateYearPHI(currentPHI: number, growthRate: number, year: number): number {
  const phiGrowthRate = growthRate * 0.3; // PHI improves at 30% of ESV rate
  return Math.min(100, currentPHI * Math.pow(1 + phiGrowthRate, year));
}
```

8.4 Cumulative Carbon Calculation

Formula:

```
Total_Carbon(year) = Σ[Carbon_Rate × (1 + (growth_rate/2) × (y-1))] for y=1 to year
```

TypeScript Implementation:

```
function calculateCumulativeCarbon(
  carbonRate: number,
  year: number,
  growthRate: number
): number {
```

```
  // Carbon accumulation increases as trees mature
  let total = 0;
  for (let y = 1; y <= year; y++) {
    total += carbonRate * (1 + (growthRate / 2) * (y - 1));
  }
  return total;
}
```

8.5 5-Year Projection Example

Scenario	Year 0 ESV	Year 5 ESV	Growth %	5yr Carbon
Conservative	$1,000	$1,104	+10.4%	8.25 tCO2
Moderate	$1,000	$1,276	+27.6%	19.25 tCO2
Optimistic	$1,000	$1,611	+61.1%	36.0 tCO2

---

9. COMPLETE FORMULA REFERENCE

9.1 Quick Reference Card

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    PPS FORMULA QUICK REFERENCE                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  NORMALIZATION (6 TYPES)                                                 │
│  ────────────────────────                                                │
│  Linear:        S = [(V - Vmin)/(Vmax - Vmin)] × 100                     │
│  Inv. Linear:   S = [(Vmax - V)/(Vmax - Vmin)] × 100                     │
│  Sigmoid:       S = 100 / [1 + exp(-k(V - Vmid))]                        │
│  Inv. Sigmoid:  S = 100 - sigmoid(V)                                     │
│  Gaussian:      S = 100 × exp[-(V - Vopt)² / (2σ²)]                      │
│  Centered:      S = 100 × [1 - (|V| / Vmax)]                             │
│                                                                          │
│  CATEGORY SCORE                                                          │
│  ──────────────                                                          │
│  Cj = Σ(wi × Si) / Σwi                                                   │
│                                                                          │
│  OVERALL PPS SCORE                                                       │
│  ──────────────────                                                      │
│  PPS = WA×A + WB×B + WC×C + WD×D + WE×E                                  │
│  (Ecosystem-adaptive weights, Σw = 1.0)                                  │
│                                                                          │
│  ESV MULTIPLIER                                                          │
│  ──────────────                                                          │
│  M = [(PPS-50)/100] × k × [1 + α × ln(PPS/50)]                           │
│  where k=0.6, α=0.15                                                     │
│                                                                          │
│  ADJUSTED ESV                                                            │
│  ────────────                                                            │
│  ESV = Baseline × (1 + M) × Regional_Factor                              │
│                                                                          │
│  CARBON CREDITS                                                          │
│  ──────────────                                                          │
```

```
│  Carbon = Biomass × 0.47                                              │
│  CO2e = Carbon × 3.67                                                 │
│  Verified = CO2e × (DQS/100)                                          │
│                                                                        │
│  DATA QUALITY SCORE                                                   │
│  ──────────────────                                                   │
│  DQS = [Σ(wi × ai) / Σwi] × 100                                      │
│  Criticality: Critical=1.0, Important=0.7, Supporting=0.4, Aux=0.2    │
│                                                                        │
│  GRADING                                                              │
│  ────────                                                             │
│  AAA: 86-100 | AA: 72-85 | A: 58-71 | BBB: 44-57                     │
│  BB: 30-43   | B: 16-29  | CCC: 0-15                                 │
│                                                                        │
│  ESV PROJECTION                                                       │
│  ───────────────                                                      │
│  ESV(y) = ESV₀ × (1 + growth)^y                                      │
│  Conservative: 2%/yr | Moderate: 5%/yr | Optimistic: 10%/yr          │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

9.2 Constants Reference

```python
# PHI-to-ESV Constants
PHI_ESV_CONSTANTS = {
    "k": 0.6,        # Base sensitivity factor
    "alpha": 0.15    # Logarithmic acceleration factor
}

# Carbon Conversion Constants
CARBON_CONSTANTS = {
    "carbon_fraction": 0.47,      # Fraction of biomass that is carbon
    "co2_to_c_ratio": 3.67,       # CO2/C molecular weight ratio (44/12)
    "root_shoot_tropical": 0.24,  # Root-to-shoot ratio for tropical forests
}

# DQS Thresholds
DQS_THRESHOLDS = {
    "category_minimum": 40,
    "overall_minimum": 50,
    "investment_grade": 70,
    "high_confidence": 85
}

# Criticality Weights
CRITICALITY_WEIGHTS = {
    "critical": 1.0,
    "important": 0.7,
    "supporting": 0.4,
    "auxiliary": 0.2
}
```

DOCUMENT INFORMATION

Source Files Referenced:

- `/Hello-/planetary_health_query/utils/normalization.py`
- `/Hello-/planetary_health_query/utils/scoring.py`
- `/Hello-/planetary_health_query/utils/quality.py`
- `/Hello-/planetary_health_query/core/config.py`
- `/src/utils/esvCalculation.ts`
- `/src/utils/esvProjection.ts`
- `/src/types/phi.ts`